COMP1661 – Mobile Application Design and Development Report

Your name	Marzuq Sogbanmu	Your Student ID	001102232

1. BRIEF STATEMENT OF FEATURES YOU HAVE IMPLEMENTED (2%)

Feature	Status	Your Comments
Functionality A	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	
Functionality B	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	
Functionality C	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	
Functionality D	Fully completed ☐ Partially completed ☐ Having bugs/Not working ☒ Not implemented ☐	Functionality implemented but the setOnQueryTextListener for the search keeps returning null.
Functionality E	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	
Functionality F	Fully completed ☒ Partially completed ☒ Having bugs/Not working ☐ Not implemented ☐	
Functionality G	Fully completed ☐ Partially completed ☒ Having bugs/Not working ☐ Not implemented ☐	The only additional feature implemented was the option to update records.

Link to recorded video (if you record your application before submitting the report) The recorded video will demonstrate the implemented product. It is roughly 10 minutes. The video you upload on Panopto should follow this naming convention – "YourName-ID", so we can quickly identify your work.
https://gre.cloud.panopto.eu/Panopto/Pages/Sessions/List.aspx#folderID="37720328-4598-476a-93b8-afd200768bb9"

2. SECTION 2 - REFLECTION (4%)

Write a reflection (approximately 350 words) on how the app was developed. Discuss lessons learnt, what you think went well and what you think could have been improved and how.

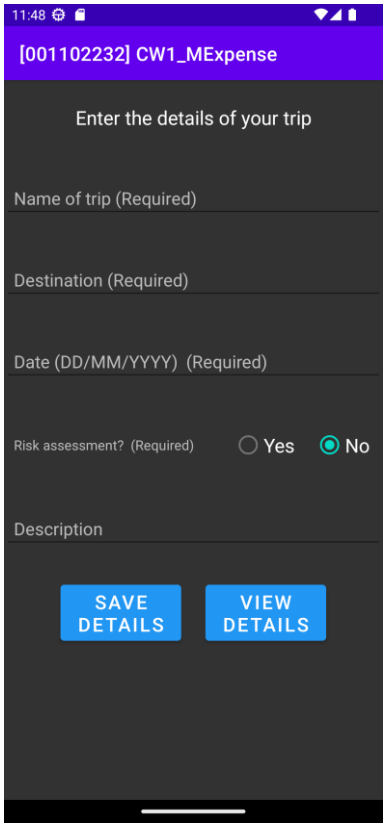
3. SECTION 3 - EVALUATION (10%):

An evaluation of your app(s). Write between 700 to 1000 words evaluating the app(s) that you have produced. Be specific and justify any statements you make. Your evaluation should include, but **need not be limited** to, the following aspects of your app: i. Human computer interaction ii. Security iii. Ability of the app to run on a range of screen sizes and how this could be improved iv. Changes that would need to be made in order for the app(s) to be deployed for live use v. Others

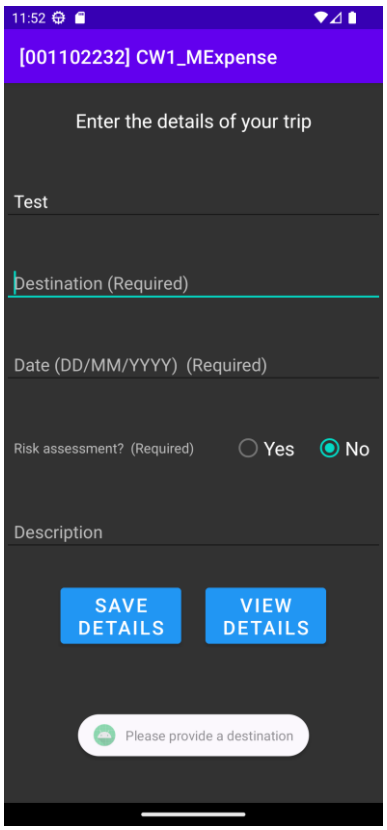
3. SECTION 4 - DESIGN (2%)

Screen shots demonstrating each of the features that you have implemented. Give captions or annotations to explain which features are being demonstrated.

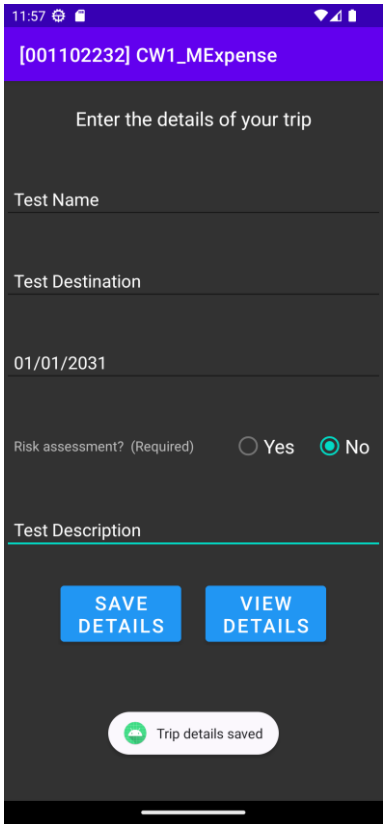
Functionality A



1 Inserting details into database.

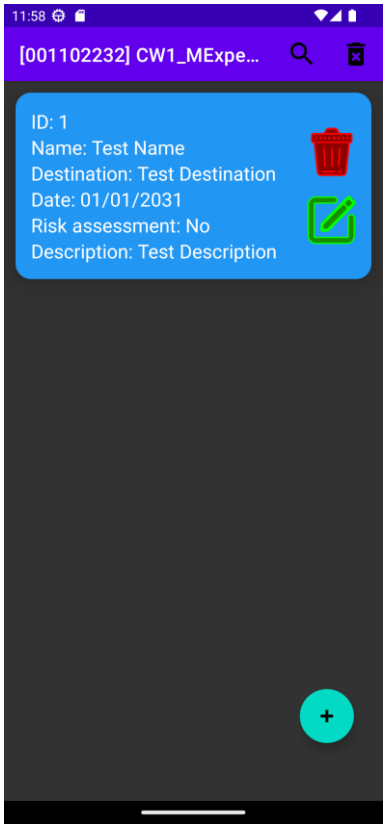


2 Validation for inserting data. The name, destination, date, and risk assessment fields are all mandatory.

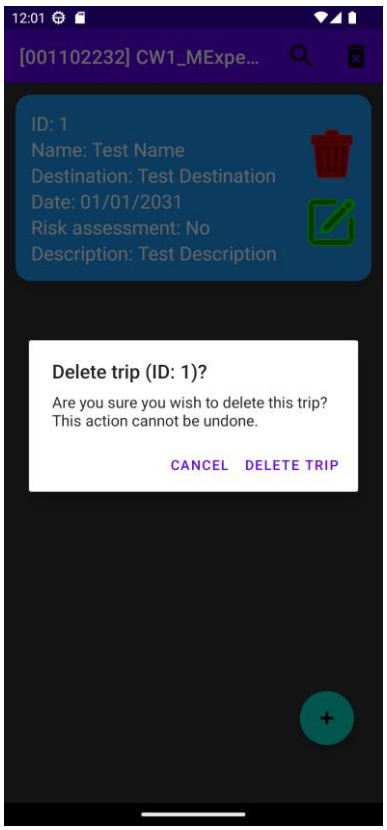


3 Confirmation of details being saved to the database.

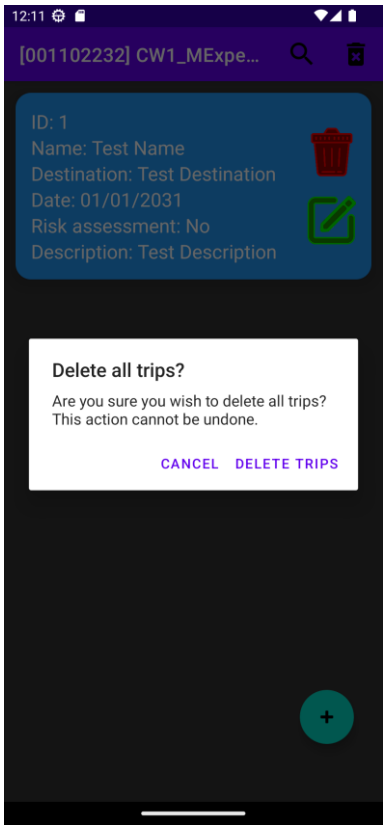
Functionality B



4 Screen for viewing all trip records in the database. Each record has buttons to delete and edit them. There is also a floating action button that will allow the user to add a new trip to the database.

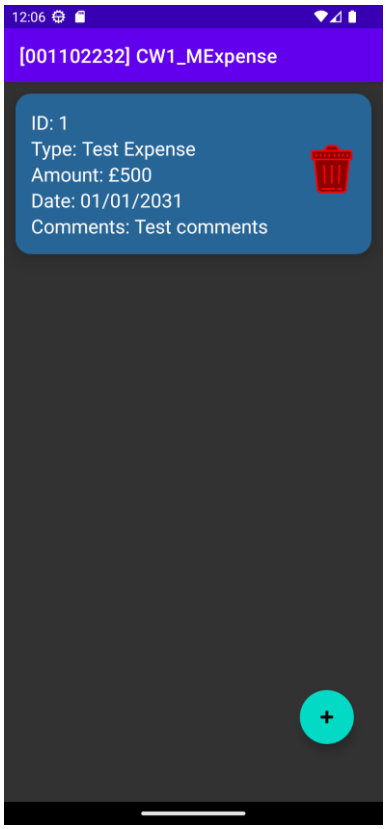


5 Deleting a record. When the delete icon is clicked, an alert dialog will appear asking the user to confirm if they wish to delete the record.

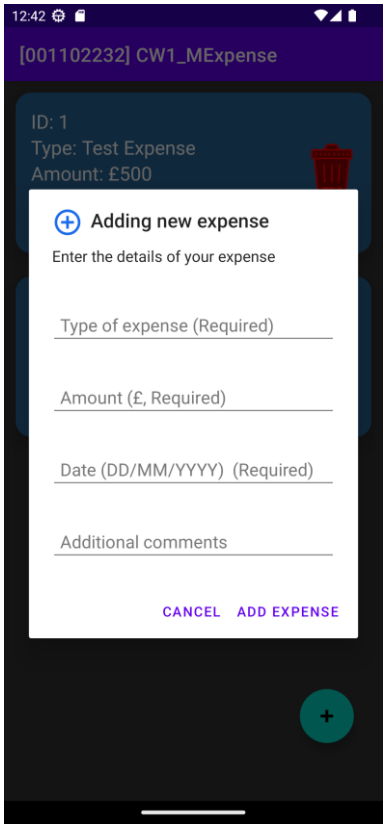


6 Deleting a record. When the delete icon in the action bar is clicked, an alert dialog will appear asking the user to confirm if they wish to delete all records in the database.

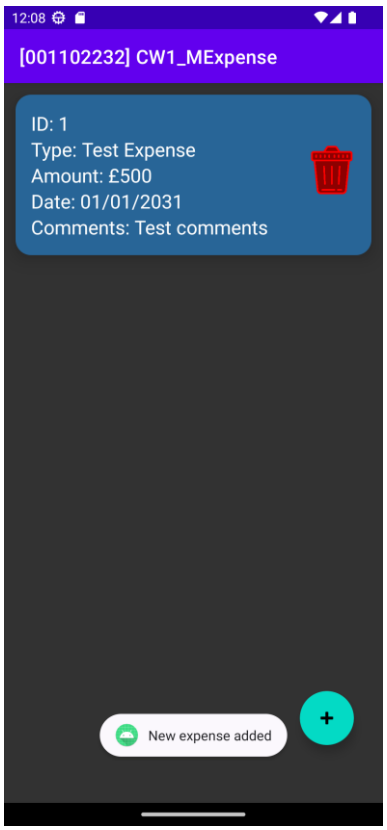
Functionality C



7 Page for viewing all expense records in the database. Each record has buttons to delete them. There is also a floating action button that will allow the user to add a new expense to the database.

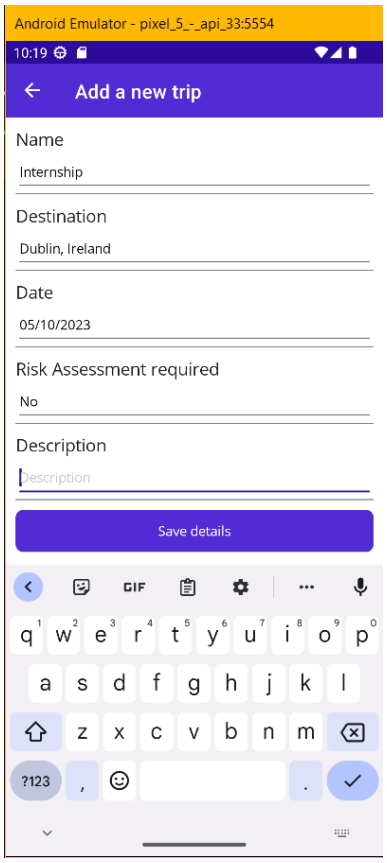


8 Inserting details of a new expense record into the database.

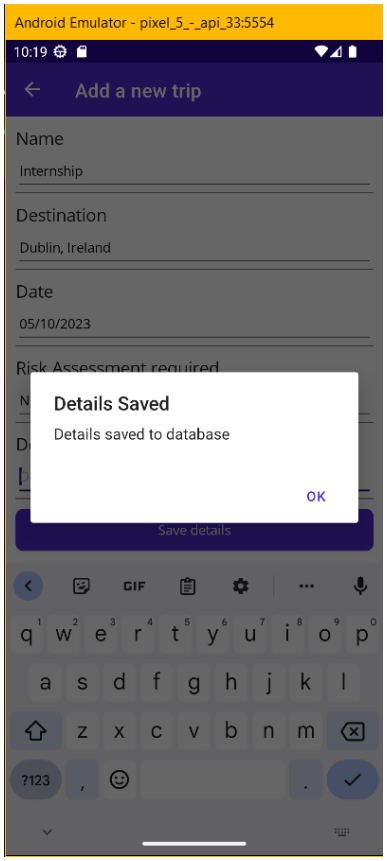


9 Confirmation of an expense record being added to the database.

Functionality E

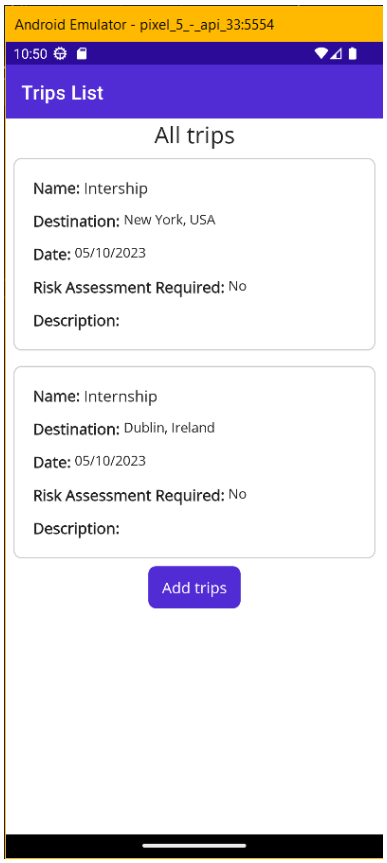


10 Inserting details into database.

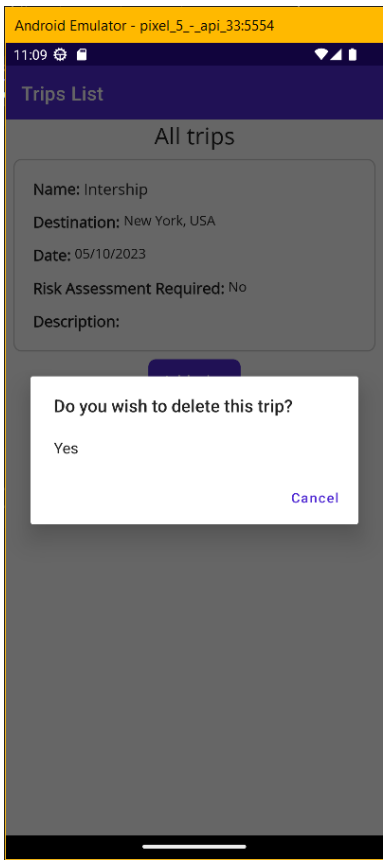


11 Confirmation of details saved.

Functionality F

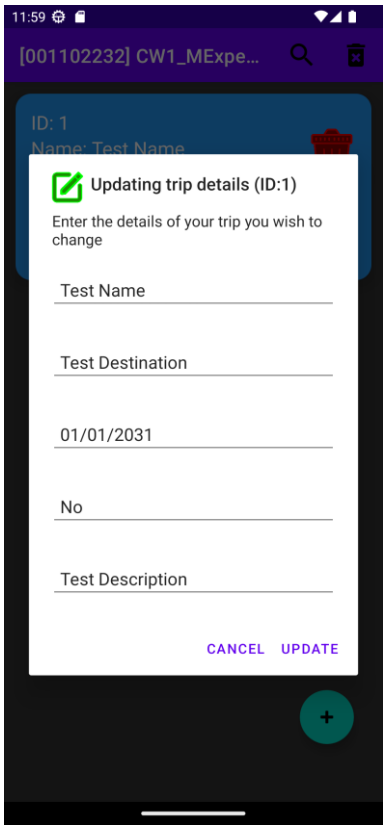


12 Viewing trips records in database.



13 Alert dialog for deleting a record.

Functionality G



14 Updating a record. Upon clicking the edit icon, an alert dialog will appear with text fields filled with the previous data from the record

4. SECTION 5 – CODE (2%)

Code listing of any code files you have written. You do not need to include generated code. Please clearly label the code, so it indicates the source file and programming language.

Java Code

DBHelper.java

```
package com.example.a001102232cw1_mexpense;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
```

```
import android.database.sqlite.SQLiteOpenHelper;
import android.util.Log;

public class DBHelper extends SQLiteOpenHelper {

    private static final String DATABASE_NAME = "M_Expense";
    private static final String table_name1 = "trips";
    public static final String col_trip_id = "trip_id";
    public static final String col_name = "trip_name";
    public static final String col_dest = "trip_dest";
    public static final String col_date = "trip_date";
    public static final String col_risk = "trip_risk";
    public static final String col_desc = "trip_desc";

    private static final String table_name2 = "expenses";
    public static final String col_exp_id = "exp_id";
    public static final String col_type = "exp_type";
    public static final String col_amount = "exp_amount";
    public static final String col_time = "exp_time";
    public static final String col_comments = "exp_comments";

    private SQLiteDatabase database;

    private static final String CREATE_TRIPS = String.format(
    "CREATE TABLE %s (" +
    " %s INTEGER PRIMARY KEY AUTOINCREMENT, " +
    " %s TEXT, " +
    " %s TEXT, " +
    " %s TEXT, " +
    " %s TEXT, " +
    " %s TEXT)",
    table_name1, col_trip_id, col_name, col_dest, col_date, col_risk, col_desc);

    private static final String CREATE_EXPENSES = String.format(
      "CREATE TABLE %s (" +
      " %s INTEGER PRIMARY KEY AUTOINCREMENT, " +
      " %s TEXT, " +
      " %s DECIMAL(3,2), " +
      " %s TEXT, " +
      " %s TEXT)",
      table_name2, col_exp_id, col_type, col_amount, col_time, col_comments);

    public DBHelper(Context context) {
        super(context, DATABASE_NAME, null, 1);
        database = getWritableDatabase();
    }

//Creation of trip and expense tables
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(CREATE_TRIPS);
        db.execSQL(CREATE_EXPENSES);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + table_name1);
        db.execSQL("DROP TABLE IF EXISTS " + table_name2);

        Log.w(this.getClass().getName(), DATABASE_NAME + " database upgrade to
version " + newVersion + ", old data lost");
        onCreate(db);
    }
```

```
//    Method to add trip data
    public Boolean trip_insert_data (String name, String dest, String date, String
risk, String desc) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put(col_name, name);
        contentValues.put(col_dest, dest);
        contentValues.put(col_date, date);
        contentValues.put(col_risk, risk);
        contentValues.put(col_desc, desc);

        long result = db.insert(table_name1, null, contentValues);

        return result != -1;
    }

//    Method to add expense data
    public Boolean exp_insert_data (String type, String amount, String time,
String comments) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put(col_type, type);
        contentValues.put(col_amount, amount);
        contentValues.put(col_time, time);
        contentValues.put(col_comments, comments);

        long result = db.insert(table_name2, null, contentValues);

        return result != -1;
    }

//    Method to retrieve trip data
    public Cursor get_tripData () {
        SQLiteDatabase db = this.getWritableDatabase();
        Cursor cursor = db.rawQuery("SELECT * FROM trips", null);
        return cursor;
    }

//    Method to retrieve expense data
    public Cursor get_expData () {
        SQLiteDatabase db = this.getWritableDatabase();
        Cursor cursor = db.rawQuery("SELECT * FROM expenses", null);
        return cursor;
    }

//    Method to search through trip names
    public Cursor search_trip(String query) {
        SQLiteDatabase db = this.getWritableDatabase();
        String[] columns = {col_trip_id, col_name, col_dest, col_date, col_risk,
col_desc};
        String selection = col_name + " LIKE ?";
        String[] selectionArgs = {"%" + query + "%"};
        Cursor cursor = db.query(table_name1, columns, selection, selectionArgs,
null, null, null);
        return cursor;
    }

//    Method to update trip data (unused)
    public boolean trip_update_data(String name, String dest, String date, String
risk, String desc) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
```

```
        contentValues.put(col_name, name);
        contentValues.put(col_dest, dest);
        contentValues.put(col_date, date);
        contentValues.put(col_risk, risk);
        contentValues.put(col_desc, desc);

        db.update(table_name1, contentValues, "trip_id =" + col_trip_id, null);
        return true;
    }
}
```

ExpenseAdapter.java

```
package com.example.a001102232cw1_mexpense;

import android.app.AlertDialog;
import android.content.Context;
import android.content.DialogInterface;
import android.database.sqlite.SQLiteDatabase;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.util.ArrayList;

public class ExpenseAdapter extends
RecyclerView.Adapter<ExpenseAdapter.ViewHolder> {
    private Context context;
    private final ArrayList exp_id, exp_type, exp_amount, exp_time, exp_comment;

    public ExpenseAdapter(Context context, ArrayList exp_id, ArrayList exp_type,
ArrayList exp_amount, ArrayList exp_time, ArrayList exp_comment) {
        this.context = context;
        this.exp_id = exp_id;
        this.exp_type = exp_type;
        this.exp_amount = exp_amount;
        this.exp_time = exp_time;
        this.exp_comment = exp_comment;
    }

    @NonNull
    @Override
    public ExpenseAdapter.ViewHolder onCreateViewHolder(@NonNull ViewGroup parent,
int viewType) {
        View v =
LayoutInflater.from(context).inflate(R.layout.expense_records,parent,false);
        return new ExpenseAdapter.ViewHolder(v);
    }

    @Override
    public void onBindViewHolder(@NonNull ExpenseAdapter.ViewHolder holder, int
position) {
        holder.exp_id.setText(String.valueOf(exp_id.get(position)));
        holder.type_id.setText(String.valueOf(exp_type.get(position)));
        holder.amount_id.setText(String.valueOf(exp_amount.get(position)));
```

```
        holder.time_id.setText(String.valueOf(exp_time.get(position)));
        holder.comment_id.setText(String.valueOf(exp_comment.get(position)));
        int exp_int = new Integer(holder.exp_id.getText().toString());

        holder.delete.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                DBHelper db = new DBHelper(view.getContext());
                SQLiteDatabase DataB = db.getWritableDatabase();
                AlertDialog.Builder builder = new
AlertDialog.Builder(view.getContext());
                builder.setTitle("Delete expense (ID: " + exp_int + ")?")
                        .setMessage("Are you sure you wish to delete this expense?
This action cannot be undone.")
                        .setPositiveButton("Delete expense", new
DialogInterface.OnClickListener() {
                            @Override
                            public void onClick(DialogInterface dialogInterface,
int i) {
                                DataB.delete("expenses", "exp_id =" + exp_int,
null);
                                if(exp_int>0) {
                                    db.get_expData();
                                    Toast.makeText(context, "Expense (ID: " +
exp_int + ") has been deleted", Toast.LENGTH_SHORT).show();
                                } else {
                                    Toast.makeText(context, "Expense (ID: " +
exp_int + ") has not been deleted", Toast.LENGTH_SHORT).show();
                                }
                            }
                        })
                        .setNegativeButton("Cancel", new
DialogInterface.OnClickListener() {
                            @Override
                            public void onClick(DialogInterface dialogInterface,
int i) {

                            }
                        });
                builder.create().show();
            }
        });
    }

    @Override
    public int getItemCount() {
        return exp_type.size();
    }

    public class ViewHolder extends RecyclerView.ViewHolder {
        LinearLayout single_trip, expenses;
        TextView exp_id;
        TextView type_id;
        TextView amount_id;
        TextView time_id;
        TextView comment_id;
        TextView trip_id, trip_name, trip_dest, trip_date, trip_risk, trip_desc;
        ImageView delete;
        public ViewHolder(@NonNull View itemView) {
            super(itemView);
            single_trip = itemView.findViewById(R.id.ll_trip);
            expenses = itemView.findViewById(R.id.ll_expenses);
            exp_id = itemView.findViewById(R.id.tv_expense_id);
```

```
            type_id = itemView.findViewById(R.id.tv_record_type);
            amount_id = itemView.findViewById(R.id.tv_record_amount);
            time_id = itemView.findViewById(R.id.tv_record_time);
            comment_id = itemView.findViewById(R.id.tv_record_comments);
            delete = itemView.findViewById(R.id.iv_exp_delete);

            trip_id = itemView.findViewById(R.id.tv_record_id);
            trip_name = itemView.findViewById(R.id.tv_record_name);
            trip_dest = itemView.findViewById(R.id.tv_record_dest);
            trip_date = itemView.findViewById(R.id.tv_record_date);
            trip_risk = itemView.findViewById(R.id.tv_record_risk);
            trip_desc = itemView.findViewById(R.id.tv_record_desc);
        }

    }
}
```

MainActivity.java

```
package com.example.a001102232cw1_mexpense;

import androidx.appcompat.app.AppCompatActivity;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.RadioButton;
import android.widget.RadioGroup;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {

    EditText name, dest, date, desc;
    RadioButton risk_buttons;
    RadioGroup risk_group;
    Button save, view;
    DBHelper db;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        name = findViewById(R.id.et_trip_name);
        dest = findViewById(R.id.et_dest);
        date = findViewById(R.id.et_date);
        desc = findViewById(R.id.et_desc);
        save = findViewById(R.id.btn_save);
        view = findViewById(R.id.btn_view);

        risk_group = (RadioGroup) findViewById(R.id.rg_risk);
        int radio_select = risk_group.getCheckedRadioButtonId();
        risk_buttons = (RadioButton) findViewById(radio_select);

        db = new DBHelper(this);

        view.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
```

```
                startActivity(new Intent(MainActivity.this, Trips.class));
            }
        });

        save.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                String name_text = name.getText().toString();
                String dest_text = dest.getText().toString();
                String date_text = date.getText().toString();
                String risk_text = risk_buttons.getText().toString();
                String desc_text = desc.getText().toString();

                if (name_text.isEmpty()) {
                    Toast.makeText(MainActivity.this, "Please provide a name",
Toast.LENGTH_SHORT).show();
                } else if (dest_text.isEmpty()) {
                    Toast.makeText(MainActivity.this, "Please provide a
destination", Toast.LENGTH_SHORT).show();
                } else if (date_text.isEmpty()) {
                    Toast.makeText(MainActivity.this, "Please provide a date",
Toast.LENGTH_SHORT).show();
                } else if (risk_text.isEmpty()) {
                    Toast.makeText(MainActivity.this, "Please check a button for
risk assessment", Toast.LENGTH_SHORT).show();
                } else {
                    Boolean check_saved_data = db.trip_insert_data(name_text,
dest_text, date_text, risk_text, desc_text);
                    if (check_saved_data==true) {
                        Toast.makeText(MainActivity.this, "Trip details saved",
Toast.LENGTH_SHORT).show();
                    } else {
                        Toast.makeText(MainActivity.this, "Failed to save
details", Toast.LENGTH_SHORT).show();
                    }
                }
            }
        });
    }
}
```

SingleTrip.java

```
package com.example.a001102232cw1_mexpense;

import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import android.annotation.SuppressLint;
import android.content.DialogInterface;
import android.content.Intent;
import android.database.Cursor;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

import com.google.android.material.floatingactionbutton.FloatingActionButton;
```

```
import java.util.ArrayList;

public class SingleTrip extends AppCompatActivity {
    RecyclerView recyclerView;
    FloatingActionButton add;
    ArrayList<String> id_no, name, name2, amount, time, comments, destination,
date, risk, description;
    DBHelper db;
    ExpenseAdapter adapter;
    TripAdapter adapter1;

    @SuppressLint("MissingInflatedId")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.recycler_single_trip);
        db = new DBHelper(this);
        id_no = new ArrayList<>();
        name = new ArrayList<>();
        amount = new ArrayList<>();
        time = new ArrayList<>();
        comments = new ArrayList<>();
        recyclerView = findViewById(R.id.rv_expenses);
        add = findViewById(R.id.fab_exp_add);
        adapter = new ExpenseAdapter(this, id_no, name, amount, time, comments);
        adapter1 = new TripAdapter(this, id_no, name2, destination, date, risk,
description);
        recyclerView.setAdapter(adapter);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        Intent intent = getIntent();
        String position = intent.getStringExtra("trip_id");
        int pos_no = Integer.parseInt(position);
        display_exp_data();
        display_trip_data();

        add.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                LayoutInflater inflater = (LayoutInflater)
SingleTrip.this.getSystemService(LAYOUT_INFLATER_SERVICE);
                View view1 = inflater.inflate(R.layout.add_expense, null);
                final EditText et_type = view1.findViewById(R.id.et_type);
                final EditText et_amount = view1.findViewById(R.id.et_amount);
                final EditText et_time = view1.findViewById(R.id.et_time);
                final EditText et_comment = view1.findViewById(R.id.et_comment);

                AlertDialog.Builder builder = new
AlertDialog.Builder(SingleTrip.this);
                builder.setView(view1)
                        .setTitle("Adding new expense")
                        .setMessage("Enter the details of your expense")
                        .setIcon(R.drawable.baseline_add_circle_outline_24)
                        .setPositiveButton("Add expense", new
DialogInterface.OnClickListener() {
                            @Override
                            public void onClick(DialogInterface dialogInterface,
int i) {
                                String type = et_type.getText().toString();
                                String amount = et_amount.getText().toString();
                                String time = et_time.getText().toString();
                                String comment = et_comment.getText().toString();
                                boolean count = db.exp_insert_data(type, amount,
time, comment);
```

```
                                if (count==true) {
                                    Toast.makeText(SingleTrip.this, "New expense
added", Toast.LENGTH_SHORT).show();
                                } else {
                                    Toast.makeText(SingleTrip.this, "Expense not
added", Toast.LENGTH_SHORT).show();
                                }
                            }
                        })
                        .setNegativeButton("Cancel", new
DialogInterface.OnClickListener() {
                            @Override
                            public void onClick(DialogInterface dialogInterface,
int i) {

                            }
                        });
                builder.create().show();
            }
        });
    }

    private void display_trip_data() {

    }

    private void display_exp_data() {
        Cursor cursor = db.get_expData();
        if (cursor.getCount()==0) {
            Toast.makeText(this, "No expense records exist",
Toast.LENGTH_SHORT).show();
            return;
        } else {
            while (cursor.moveToNext()) {
                id_no.add(cursor.getString(0));
                name.add(cursor.getString(1));
                amount.add(cursor.getString(2));
                time.add(cursor.getString(3));
                comments.add(cursor.getString(4));
            }
        }
    }
}
```

TripAdapter.java

```
package com.example.a001102232cw1_mexpense;

import android.app.AlertDialog;
import android.content.ContentValues;
import android.content.Context;
import android.content.DialogInterface;
import android.content.Intent;
import android.database.sqlite.SQLiteDatabase;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;
```

```
import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.util.ArrayList;

public class TripAdapter extends RecyclerView.Adapter<TripAdapter.ViewHolder> {
    private Context context;
    private ArrayList trip_id, name_id, dest_id, date_id, risk_id, desc_id;

    DBHelper db;

    public TripAdapter(Context context, ArrayList trip_id, ArrayList name_id,
ArrayList dest_id, ArrayList date_id, ArrayList risk_id, ArrayList desc_id) {
        this.context = context;
        this.trip_id = trip_id;
        this.name_id = name_id;
        this.dest_id = dest_id;
        this.date_id = date_id;
        this.risk_id = risk_id;
        this.desc_id = desc_id;
    }

    @NonNull
    @Override
    public TripAdapter.ViewHolder onCreateViewHolder(@NonNull ViewGroup parent,
int viewType) {
        View v =
LayoutInflater.from(context).inflate(R.layout.trip_records,parent,false);
        return new ViewHolder(v);
    }

    @Override
    public void onBindViewHolder(@NonNull TripAdapter.ViewHolder holder, int
position) {
        holder.trip_id.setText(String.valueOf(trip_id.get(position)));
        holder.name_id.setText(String.valueOf(name_id.get(position)));
        holder.dest_id.setText(String.valueOf(dest_id.get(position)));
        holder.date_id.setText(String.valueOf(date_id.get(position)));
        holder.risk_id.setText(String.valueOf(risk_id.get(position)));
        holder.desc_id.setText(String.valueOf(desc_id.get(position)));
        int trip_int = new Integer(holder.trip_id.getText().toString());

        holder.delete.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                DBHelper db = new DBHelper(view.getContext());
                SQLiteDatabase DataB = db.getWritableDatabase();
                AlertDialog.Builder builder = new
AlertDialog.Builder(view.getContext());
                builder.setTitle("Delete trip (ID: " + trip_int + ")?")
                    .setMessage("Are you sure you wish to delete this trip? This
action cannot be undone.")
                    .setPositiveButton("Delete trip", new
DialogInterface.OnClickListener() {
                        @Override
                        public void onClick(DialogInterface dialogInterface, int
i) {
                            DataB.delete("trips", "trip_id =" + trip_int, null);
                            notifyDataSetChanged();
                            if(trip_int>0) {
                                db.get_tripData();
                                Toast.makeText(context, "Trip (ID: " + trip_int +
```

```
") has been deleted", Toast.LENGTH_SHORT).show();
                            } else {
                                Toast.makeText(context, "Trip (ID: " + trip_int +
") has not been deleted", Toast.LENGTH_SHORT).show();
                            }
                        }
                    })
                    .setNegativeButton("Cancel", new
DialogInterface.OnClickListener() {
                        @Override
                        public void onClick(DialogInterface dialogInterface, int
i) {

                        }
                    });
                builder.create().show();
            }
        });

        holder.edit.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                LayoutInflater inflater = (LayoutInflater)
context.getSystemService(Context.LAYOUT_INFLATER_SERVICE);
                View view1 = inflater.inflate(R.layout.edit_trip, null);
                DBHelper db = new DBHelper(view1.getContext());
                SQLiteDatabase DataB = db.getWritableDatabase();
                final EditText edit_name = view1.findViewById(R.id.et_edit_name);
                final EditText edit_dest = view1.findViewById(R.id.et_edit_dest);
                final EditText edit_date = view1.findViewById(R.id.et_edit_date);
                final EditText edit_risk = view1.findViewById(R.id.et_edit_risk);
                final EditText edit_desc = view1.findViewById(R.id.et_edit_desc);
                String old_name = holder.name_id.getText().toString();
                String old_dest = holder.dest_id.getText().toString();
                String old_date = holder.date_id.getText().toString();
                String old_risk = holder.risk_id.getText().toString();
                String old_desc = holder.desc_id.getText().toString();
                edit_name.setText(old_name);
                edit_dest.setText(old_dest);
                edit_date.setText(old_date);
                edit_risk.setText(old_risk);
                edit_desc.setText(old_desc);

                androidx.appcompat.app.AlertDialog.Builder builder = new
androidx.appcompat.app.AlertDialog.Builder(view1.getContext());
                builder.setView(view1)
                        .setTitle("Updating trip details (ID:" + trip_int + ")")
                        .setMessage("Enter the details of your trip you wish to
change")
                        .setIcon(R.drawable.edit_pencil)
                        .setPositiveButton("Update", new
DialogInterface.OnClickListener() {
                            @Override
                            public void onClick(DialogInterface dialogInterface,
int i) {
                                String new_name = edit_name.getText().toString();
                                String new_dest = edit_dest.getText().toString();
                                String new_date = edit_date.getText().toString();
                                String new_risk = edit_risk.getText().toString();
                                String new_desc = edit_desc.getText().toString();
                                ContentValues contentValues = new ContentValues();
                                contentValues.put("trip_name", new_name);
                                contentValues.put("trip_dest", new_dest);
```

```
                                contentValues.put("trip_date", new_date);
                                contentValues.put("trip_risk", new_risk);
                                contentValues.put("trip_desc", new_desc);
                                DataB.update("trips", contentValues, "trip_id =" +
trip_int, null);
//                                  boolean count = db.trip_update_data(name, dest,
date, risk, desc);
                                notifyDataSetChanged();
                                if (trip_int>0) {
                                    Toast.makeText(view1.getContext(), "Details
for trip (ID: " + trip_int + ") updated", Toast.LENGTH_SHORT).show();
                                } else {
                                    Toast.makeText(view1.getContext(), "Details
not updated", Toast.LENGTH_SHORT).show();
                                }
                            }
                        })
                        .setNegativeButton("Cancel", new
DialogInterface.OnClickListener() {
                            @Override
                            public void onClick(DialogInterface dialogInterface,
int i) {

                            }
                        });
                builder.create().show();
            }
        });

//        OnClickListener to open new
        holder.single_trip.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Intent intent = new Intent(context, SingleTrip.class);
                intent.putExtra("trip_id", String.valueOf(trip_int));
                context.startActivity(intent);
            }
        });

    }

    @Override
    public int getItemCount() {
        return name_id.size();
    }

//    Arraylist of filtered fields (for search function)
    public void filterList(ArrayList<String> filteredIdNo, ArrayList<String>
filteredName, ArrayList<String> filteredDestination, ArrayList<String>
filteredDate, ArrayList<String> filteredRisk, ArrayList<String>
filteredDescription) {
        trip_id = filteredIdNo;
        name_id = filteredName;
        dest_id = filteredDestination;
        date_id = filteredDate;
        risk_id = filteredRisk;
        desc_id = filteredDescription;
        notifyDataSetChanged();
    }


    public class ViewHolder extends RecyclerView.ViewHolder {
        LinearLayout single_trip;
```

```
        TextView trip_id, name_id, dest_id, date_id, risk_id, desc_id;
        ImageView edit, delete;
        public ViewHolder(View itemView) {
            super(itemView);
            single_trip = itemView.findViewById(R.id.ll_trip);
            trip_id = itemView.findViewById(R.id.tv_record_id);
            name_id = itemView.findViewById(R.id.tv_record_name);
            dest_id = itemView.findViewById(R.id.tv_record_dest);
            date_id = itemView.findViewById(R.id.tv_record_date);
            risk_id = itemView.findViewById(R.id.tv_record_risk);
            desc_id = itemView.findViewById(R.id.tv_record_desc);
            edit = itemView.findViewById(R.id.iv_edit);
            delete = itemView.findViewById(R.id.iv_delete);
        }
    }
}
```

Trips.java

```
package com.example.a001102232cw1_mexpense;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.view.MenuItemCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import android.app.AlertDialog;
import android.content.DialogInterface;
import android.content.Intent;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.SearchView;
import android.widget.Toast;

import com.google.android.material.floatingactionbutton.FloatingActionButton;

import java.util.ArrayList;

public class Trips extends AppCompatActivity implements
SearchView.OnQueryTextListener{
    RecyclerView recyclerView;
    FloatingActionButton add;
    ArrayList<String> id_no, name, destination, date, risk, description;
    DBHelper db;
    TripAdapter adapter;
    SearchView searchView;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.recycler_all_trips);
        db = new DBHelper(this);
        id_no = new ArrayList<>();
        name = new ArrayList<>();
        destination = new ArrayList<>();
        date = new ArrayList<>();
        risk = new ArrayList<>();
```

```
        description = new ArrayList<>();
        recyclerView = findViewById(R.id.rv_trips);
        add = findViewById(R.id.fab_add);
        adapter = new TripAdapter(this, id_no, name, destination, date, risk,
description);
        recyclerView.setAdapter(adapter);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        display_data();

        add.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                startActivity(new Intent(Trips.this, MainActivity.class));
            }
        });
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.trips_options, menu);
        MenuItem search = menu.findItem(R.id.search);
//        searchView = (SearchView) MenuItemCompat.getActionView(search);
//        searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener);
        return super.onCreateOptionsMenu(menu);
    }

    @Override
    public boolean onOptionsItemSelected(@NonNull MenuItem item) {
        if (item.getTitle().equals("delete")) {
            SQLiteDatabase DataB = db.getWritableDatabase();
            AlertDialog.Builder builder = new AlertDialog.Builder(this);
            builder.setTitle("Delete all trips?")
                    .setMessage("Are you sure you wish to delete all trips? This
action cannot be undone.")
                    .setPositiveButton("Delete trips", new
DialogInterface.OnClickListener() {
                        @Override
                        public void onClick(DialogInterface dialogInterface, int
i) {
                            int deleted_trips = DataB.delete("trips", "1", null);
                            finish();
                            startActivity(getIntent());
                            Toast.makeText(Trips.this, deleted_trips + " trips
have been deleted", Toast.LENGTH_SHORT).show();
                        }
                    })
                    .setNegativeButton("Cancel", new
DialogInterface.OnClickListener() {
                        @Override
                        public void onClick(DialogInterface dialogInterface, int
i) {

                        }
                    });
            builder.create().show();
        } else {

        }

        return super.onOptionsItemSelected(item);
    }

    private void display_data() {
```

```
        Cursor cursor = db.get_tripData();
        if (cursor.getCount()==0) {
            Toast.makeText(this, "No trip records exist",
Toast.LENGTH_SHORT).show();
            return;
        } else {
            while (cursor.moveToNext()) {
                id_no.add(cursor.getString(0));
                name.add(cursor.getString(1));
                destination.add(cursor.getString(2));
                date.add(cursor.getString(3));
                risk.add(cursor.getString(4));
                description.add(cursor.getString(5));
            }
        }
    }

    @Override
    public boolean onQueryTextSubmit(String query) {
        return false;
    }

    @Override
    public boolean onQueryTextChange(String newText) {
        newText = newText.toLowerCase();
        ArrayList<String> filteredIdNo = new ArrayList<>();
        ArrayList<String> filteredName = new ArrayList<>();
        ArrayList<String> filteredDestination = new ArrayList<>();
        ArrayList<String> filteredDate = new ArrayList<>();
        ArrayList<String> filteredRisk = new ArrayList<>();
        ArrayList<String> filteredDescription = new ArrayList<>();

        Cursor cursor = db.search_trip(newText);

        while (cursor.moveToNext()) {
            filteredIdNo.add(cursor.getString(0));
            filteredName.add(cursor.getString(1));
            filteredDestination.add(cursor.getString(2));
            filteredDate.add(cursor.getString(3));
            filteredRisk.add(cursor.getString(4));
            filteredDescription.add(cursor.getString(5));
        }

        adapter.filterList(filteredIdNo, filteredName, filteredDestination,
filteredDate, filteredRisk, filteredDescription);
        return true;
    }

}
```

Xamarin Code

TripModel.cs

```
using SQLite;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI.Models
```

```
{
    public class TripsModel
    {
        [PrimaryKey, AutoIncrement] public int TripID { get; set; }

        public string Name { get; set;}

        public string Destination { get; set; }

        public string Date { get; set; }

        public string Risk { get; set; }

        public string Description { get; set;}
    }
}
```

Int_TripService.cs

```
using _001102232__COMP1661_CW1_M_Expense_MAUI.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI.Services
{
    public interface Int_TripsService
    {
        Task<List<TripsModel>> GetTrips();
        Task<int> AddTrip(TripsModel tripsModel);
        Task<int> DeleteTrip(TripsModel tripsModel);
    }
}
```

TripService.cs

```
using _001102232__COMP1661_CW1_M_Expense_MAUI.Models;
using SQLite;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI.Services
{
    internal class TripsService : Int_TripsService
    {
        private SQLiteAsyncConnection _dbconnect;
        public TripsService()
        {
            DB_setup();
        }
        private async void DB_setup()
        {
            if (_dbconnect == null)
            {
                string dbPath =
Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData),
"Trips.db");
                _dbconnect = new SQLiteAsyncConnection(dbPath);
```

```
                await _dbconnect.CreateTableAsync<TripsModel>();
            }
        }

        public Task<int> AddTrip(TripsModel tripsModel)
        {
           return _dbconnect.InsertAsync(tripsModel);
        }

        public Task<int> DeleteTrip(TripsModel tripsModel)
        {
            return _dbconnect.DeleteAsync(tripsModel);
        }

        public Task<List<TripsModel>> GetTrips()
        {
            var trips_list = _dbconnect.Table<TripsModel>().ToListAsync();
            return trips_list;
        }
    }
}
```

AddTripsViewModel.cs

```
using _001102232__COMP1661_CW1_M_Expense_MAUI.Services;
using CommunityToolkit.Mvvm.ComponentModel;
using Microsoft.Toolkit.Mvvm.Input;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI.ViewModels
{
    public partial class AddTripsViewModel : ObservableObject
    {
        private readonly Int_TripsService _tripsService;
        public AddTripsViewModel(Int_TripsService tripsService)
        {
            _tripsService = tripsService;
        }

        [ObservableProperty]
        private string _name;

        [ObservableProperty]
        private string _destination;

        [ObservableProperty]
        private string _date;

        [ObservableProperty]
        private string _risk;

        [ObservableProperty]
        private string _description;

        [ICommand]
        public async void Add_Trip()
        {
            var confirm = await _tripsService.AddTrip(new Models.TripsModel
            {
                Name = Name,
```

```
                Destination = Destination,
                Date = Date,
                Risk = Risk,
                Description = Description
            });

            if (confirm > 0)
            {
                await Shell.Current.DisplayAlert("Details Saved", "Details saved to
database", "OK");
            } else
            {
                await Shell.Current.DisplayAlert("Details Not Saved", "Details failed to
be saved to database", "OK");
            }
        }
    }
}
```

TripListViewModel.cs

```
using _001102232__COMP1661_CW1_M_Expense_MAUI.Models;
using _001102232__COMP1661_CW1_M_Expense_MAUI.Services;
using _001102232__COMP1661_CW1_M_Expense_MAUI.Views;
using CommunityToolkit.Mvvm.ComponentModel;
using Microsoft.Toolkit.Mvvm.Input;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI.ViewModels
{
    public partial class TripListViewModel : ObservableObject
    {
        public ObservableCollection<TripsModel> Trips { get; set; } = new
ObservableCollection<TripsModel>();

        private readonly Int_TripsService _tripsService;
        public TripListViewModel(Int_TripsService tripsService)
        {
            _tripsService = tripsService;
        }

        [ICommand]
        public async void GetTripList()
        {
            var triplist = await _tripsService.GetTrips();

            if (triplist.Count > 0) {
                Trips.Clear();
                foreach(var trip in triplist)
                {
                    Trips.Add(trip);
                }
            }
        }

        [ICommand]
        public async void Add_Trip()
        {
            await AppShell.Current.GoToAsync(nameof(AddTrips));
```

```
        }

        [ICommand]
        public async void ActionDisplay(TripsModel tripsModel)
        {
            var confirm = await AppShell.Current.DisplayActionSheet("Do you wish to
delete this trip?", "Cancel", null, "Yes");
            if (confirm != null)
            {
               var delete_confirm = await _tripsService.DeleteTrip(tripsModel);
                if (delete_confirm > 0)
                {
                    GetTripList();
                }
            }
        }
    }
}
```

AddTrips.xaml

```
using _001102232__COMP1661_CW1_M_Expense_MAUI.ViewModels;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI.Views;

public partial class AddTrips : ContentPage
{
       public AddTrips(AddTripsViewModel viewModel)
       {
              InitializeComponent();
              this.BindingContext = viewModel;
       }
}
```

AddTrips.xaml.cs

```
using _001102232__COMP1661_CW1_M_Expense_MAUI.ViewModels;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI.Views;

public partial class AddTrips : ContentPage
{
       public AddTrips(AddTripsViewModel viewModel)
       {
              InitializeComponent();
              this.BindingContext = viewModel;
       }
}
```

TripList.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:models="clr-namespace:_001102232__COMP1661_CW1_M_Expense_MAUI.Models"
             x:DataType="viewModels:TripListViewModel"
             xmlns:viewModels="clr-
namespace:_001102232__COMP1661_CW1_M_Expense_MAUI.ViewModels"
             x:Class="_001102232__COMP1661_CW1_M_Expense_MAUI.Views.TripList"
             Title="Trips List">
    <VerticalStackLayout>
        <Label
            Text="All trips"
            FontSize="24"
            HorizontalOptions="Center" />
```

```
        <CollectionView ItemsSource="{Binding Trips}">
            <CollectionView.ItemTemplate>
                <DataTemplate x:DataType="models:TripsModel">
                    <Frame  Margin="8">
                        <StackLayout Spacing="12">
                            <HorizontalStackLayout>
                                <Label Text="Name: " FontSize="16"
FontAttributes="Bold"/>
                                <Label Text="{Binding Name}" FontSize="16"/>
                            </HorizontalStackLayout>

                            <HorizontalStackLayout>
                                <Label Text="Destination: " FontSize="16"
FontAttributes="Bold"/>
                                <Label Text="{Binding Destination}"/>
                            </HorizontalStackLayout>

                            <HorizontalStackLayout>
                                <Label Text="Date: " FontSize="16"
FontAttributes="Bold"/>
                                <Label Text="{Binding Date}"/>
                            </HorizontalStackLayout>

                            <HorizontalStackLayout>
                                <Label Text="Risk Assessment Required: " FontSize="16"
FontAttributes="Bold"/>
                                <Label Text="{Binding Risk}"/>
                            </HorizontalStackLayout>

                            <HorizontalStackLayout>
                                <Label Text="Description: " FontSize="16"
FontAttributes="Bold"/>
                                <Label Text="{Binding Description}"/>
                            </HorizontalStackLayout>
                        </StackLayout>
                        <Frame.GestureRecognizers>
                            <TapGestureRecognizer Command="{Binding
Source={x:RelativeSource AncestorType={x:Type viewModels:TripListViewModel}},
Path=ActionDisplayCommand}"
                                                  CommandParameter="{Binding .}"/>
                        </Frame.GestureRecognizers>
                    </Frame>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>

        <Button
            Text="Add trips"
            FontSize="16"
            HorizontalOptions="Center"
            Command="{Binding Add_TripCommand}"/>
    </VerticalStackLayout>
</ContentPage>
```

TripList.xaml.cs

```
using _001102232__COMP1661_CW1_M_Expense_MAUI.ViewModels;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI.Views;

public partial class TripList : ContentPage
{
```

```
    private TripListViewModel _viewmode;
       public TripList(TripListViewModel viewModel)
       {
         InitializeComponent();
         _viewmode = viewModel;
              this.BindingContext = viewModel;
       }

    protected override void OnAppearing()
    {
        base.OnAppearing();
        _viewmode.GetTripListCommand.Execute(null);
    }
}
```

AppShell.xaml

```
<?xml version="1.0" encoding="UTF-8" ?>
<Shell
    x:Class="_001102232__COMP1661_CW1_M_Expense_MAUI.AppShell"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:views="clr-namespace:_001102232__COMP1661_CW1_M_Expense_MAUI.Views"
    Shell.FlyoutBehavior="Disabled">

    <ShellContent
        Title="Home"
        ContentTemplate="{DataTemplate views:TripList}"
        Route="MainPage" />

</Shell>
```

AppShell.xaml.cs

```
using _001102232__COMP1661_CW1_M_Expense_MAUI.Views;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI;

public partial class AppShell : Shell
{
       public AppShell()
       {
              InitializeComponent();
              Routing.RegisterRoute(nameof(AddTrips), typeof(AddTrips));
       }
}
```

MauiProgram.cs

```
using _001102232__COMP1661_CW1_M_Expense_MAUI.Services;
using _001102232__COMP1661_CW1_M_Expense_MAUI.ViewModels;
using _001102232__COMP1661_CW1_M_Expense_MAUI.Views;
using Microsoft.Extensions.Logging;

namespace _001102232__COMP1661_CW1_M_Expense_MAUI;

public static class MauiProgram
{
       public static MauiApp CreateMauiApp()
       {
              var builder = MauiApp.CreateBuilder();
              builder
                     .UseMauiApp<App>()
                     .ConfigureFonts(fonts =>
                     {
```

```
                    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                    fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
                });

            //Registering views
            builder.Services.AddSingleton<TripList>();
            builder.Services.AddSingleton<AddTrips>();

            //Registering viewmodels
        builder.Services.AddSingleton<TripListViewModel>();
            builder.Services.AddSingleton<AddTripsViewModel>();

        //Registering services
        builder.Services.AddSingleton<Int_TripsService, TripsService>();

#if DEBUG
        builder.Logging.AddDebug();
#endif

            return builder.Build();
      }
}
```